



INSTITUTO FEDERAL
Brasília



Lista de Exercícios 2

1. Implemente um sistema de gerenciamento de cursos e alunos em Python, utilizando orientação a objetos. O sistema deve funcionar por meio de um menu interativo e permitir as seguintes operações:

Listar todos os alunos cadastrados.

Listar todos os cursos cadastrados.

Matricular um aluno em um curso.

Exibir em quais cursos um aluno está matriculado.

Encerrar o programa.

Os alunos e cursos já devem estar previamente cadastrados no início do programa. O usuário poderá escolher as opções pelo menu, realizar matrículas e consultar as informações desejadas. Cada aluno pode se matricular em vários cursos, e cada curso pode ter vários alunos.

2. Implemente um sistema bancário em Python, utilizando os conceitos de Programação Orientada a Objetos, conforme o exemplo abaixo. O sistema deve simular operações bancárias como criação de contas (corrente e poupança), depósitos, saques, transferências, alteração e histórico de titularidade, além do gerenciamento de chave Pix.

O sistema deve possuir as seguintes características:

Cadastro de clientes (pessoa física ou jurídica) e contas bancárias.

Depósito, saque e transferência entre contas (exceto para contas poupança, que não permitem transferências).

Alteração do nome do titular da conta, com registro do histórico de alterações (data, nome antigo e novo).

Consulta do histórico de titularidade, exibindo o número da conta, o titular atual, CPF/CNPJ e a chave Pix cadastrada.

Gerenciamento de chave Pix: cadastrar e alterar a chave Pix, validando conforme os padrões do Banco Central (CPF, CNPJ, e-mail, telefone ou chave aleatória UUID). O código de validação deve citar as referências utilizadas para os padrões de regex.

Exibição do extrato da conta, incluindo todas as transações realizadas e as principais taxas bancárias.

Busca de contas por número, nome do titular ou CPF/CNPJ, inclusive para alteração de titularidade ou limite.

Interface de menu para interação com o usuário, permitindo executar todas as operações acima.

Exemplo de execução esperada:

```
Menu:
1 - Listar contas
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
6 - Criar nova conta
7 - Alterar titular da conta (por número, nome ou CPF/CNPJ)
8 - Ver histórico de titularidade
9 - Alterar limite da conta corrente
10 - Cadastrar chave Pix
11 - Alterar chave Pix
12 - Sair

Escolha uma opção: 10
Número da conta: 2020
Digite a chave Pix para cadastrar: 123.456.789-09
Chave Pix '123.456.789-09' cadastrada com sucesso para a conta 2020.
```

3. Você foi contratado para desenvolver um sistema de empréstimos e reservas de recursos para uma instituição de ensino, utilizando Python com Programação Orientada a Objetos (POO). A plataforma deve atender aos seguintes requisitos:

Recursos

Crie uma classe abstrata Recurso, com subclasses: Livro, KitRobotica e Notebook.

Cada tipo deve conter atributos específicos (ex: autor/ano, componentes, especificações).

Empréstimo de Notebook exige aceite de termo de responsabilidade.

Usuários

Crie uma classe base Usuario, com subclasses: Aluno, Servidor e Visitante, com diferentes limites de empréstimos. Todos os usuários devem ter nome, matrícula e e-mail. Permita alterar nome e e-mail via menu.

Empréstimos e Reservas

Implemente classes Emprestimo e Reserva, com controle de datas, status e histórico por usuário e recurso.

Permita renovação de empréstimos.

Regras de Negócio

Valide: limites de empréstimo, disponibilidade, conflitos de reserva e políticas (ex: termo assinado).

Use exceções personalizadas para tratar regras violadas.

Notificações

Crie notificadores para console e e-mail.

Registre a última mensagem enviada por e-mail ao realizar um empréstimo.

Menu Interativo

Inclua as opções:

Inclua as opções:

Listar recursos

Cadastrar usuário

Realizar empréstimo / devolver / renovar

Reservar recurso

Consultas (histórico por usuário/recurso, usuários cadastrados, última mensagem de e-mail)

Alterar dados do usuário

Sair

Documentação

Comente o código explicando estruturas Python e os conceitos de POO aplicados.

Use: classes abstratas (ABC), herança, polimorfismo, encapsulamento, agregação/composição e exceções personalizadas.

Com este exercício, você irá praticar de forma integrada os principais conceitos de POO em Python, desenvolvendo um sistema completo, robusto e didático, com menu interativo e código bem comentado.

Objetivo:

Com este exercício, você irá praticar de forma integrada os principais conceitos de POO em Python, desenvolvendo um sistema completo, robusto e didático, com menu interativo e código bem comentado.

Exemplos de saídas esperadas:

Menu interativo:

```
Plataforma de Empréstimos Acadêmicos (PEA)
Escolha uma opção:
1 - Listar recursos
2 - Cadastrar usuário
3 - Realizar empréstimo
4 - Devolver recurso
5 - Reservar recurso
6 - Consultas
7 - Alterar informações dos usuários
8 - Renovar empréstimo
9 - Sair
Digite sua opção: 
```

Ao realizar empréstimos:

```
===== NOTIFICAÇÃO POR E-MAIL =====
Empréstimo realizado!
Recurso: Livro: POO em Python (2022) - Autor: Ana Silva
Usuário: Carlos
Duração do empréstimo: 7 dia(s), de 30/08/2025 até 06/09/2025.
=====

===== NOTIFICAÇÃO NO CONSOLE =====
Empréstimo realizado!
Recurso: Livro: POO em Python (2022) - Autor: Ana Silva
Usuário: Carlos
Duração do empréstimo: 7 dia(s), de 30/08/2025 até 06/09/2025.
=====

Empréstimo realizado com sucesso!
```

Ao renovar empréstimos:

```
Escolha o empréstimo para renovar: 1
Empréstimo renovado com sucesso! Novo prazo: 14 dias.
```

Ao realizar consultas:

```
Plataforma de Empréstimos Acadêmicos (PEA)
Escolha uma opção:
1 - Listar recursos
2 - Cadastrar usuário
3 - Realizar empréstimo
4 - Devolver recurso
5 - Reservar recurso
6 - Consultas
7 - Alterar informações dos usuários
8 - Renovar empréstimo
9 - Sair
Digite sua opção: 6

=== CONSULTAS ===
1 - Histórico de empréstimos de um recurso
2 - Histórico de reservas de um recurso
3 - Histórico de empréstimos de um usuário
4 - Histórico de reservas de um usuário
5 - Consultar última mensagem enviada por e-mail ao realizar empréstimo
6 - Consultar usuários cadastrados
7 - Voltar ao menu principal
Escolha uma opção de consulta: █
```

4. Sistema de Votação Eletrônica com Anonimização e Persistência.

Requisitos:

1. Estrutura do sistema

Crie as classes Candidato, Eleitor e Eleicao.

Cada candidato possui nome e contagem de votos.

O eleitor informa seu CPF, mas o sistema armazena apenas o hash do CPF para garantir anonimato.

2. Fluxo de votação

O menu principal deve apresentar as opções:

1. Votar

2. Resultado da Eleição

3. Sair

Ao votar, o eleitor informa seu CPF e escolhe um candidato.

O sistema impede que o mesmo eleitor vote mais de uma vez.

Caso o eleitor tente votar novamente, deve aparecer:

Este eleitor já votou.

Caso o candidato não exista, deve aparecer:

Candidato não encontrado.

3. Persistência

Os votos devem ser salvos em um arquivo JSON na pasta Downloads do usuário, de forma anonimizada (apenas hash do CPF e apuração dos votos).

Ao iniciar o programa, os votos anteriores devem ser carregados automaticamente.

4. Apuração

O sistema deve exibir o resultado da eleição, mostrando a quantidade de votos e o percentual de cada candidato.

Em caso de empate, o sistema deve informar quais candidatos empataram.

Exemplo de saída esperada:

```
1 - Votar
2 - Resultado
3 - Sair
Escolha: 1
Digite seu CPF: 12345678900
Candidatos:
- Alice
- Bob
- Carol
Digite o nome do candidato: Alice
Voto registrado com sucesso!

1 - Votar
2 - Resultado
3 - Sair
Escolha: 1
Digite seu CPF: 12345678911
Candidatos:
- Alice
- Bob
- Carol
Digite o nome do candidato: Bob
Voto registrado com sucesso!

1 - Votar
2 - Resultado
3 - Sair
Escolha: 2

Resultado da Eleição:
Alice: 1 voto(s) (50.00%)
Bob: 1 voto(s) (50.00%)
Carol: 0 voto(s) (0.00%)

Empate entre: Alice, Bob
```

Observações:

O sistema não deve exibir o CPF real do eleitor em nenhum momento.

O arquivo de votos deve ser salvo automaticamente e carregado ao iniciar o programa.

O sistema deve ser robusto para não perder votos em caso de interrupção inesperada.